

Lab 06 — Your First Triangle

CP479 / 2110479 Computer Graphics · Lab 06 report

Viritphon Chongpermwattanol (Kin) · Computer Engineering, Chulalongkorn University

Still to fill in. Every value written `{...}` is a measurement that only exists on my machine, and every `{screenshot}` a panel I have to capture. The derivations, invocation counts and closed-form results below are complete and independent of the hardware.

Needed	From
<code>{screenshot: info}</code> , device name, subgroup size, timestamp period	<code>make info</code>
<code>{screenshot: part I}</code> , centre-pixel RGB, coverage, gpu time	<code>make triangle</code>
six cells × three loop counts	<code>make cost</code>
eight stripe widths + the no-branch run	<code>make stripes</code>
2 × 2 table of four variants	<code>make earlyz</code>
byte panel, coverage, gpu time	<code>make index</code>
three scenes × four sweeps, and the 5000-draws / 1-draw pair	<code>make probe</code>
one frame capture, three most expensive events	Xcode GPU trace

0. The machine

Every number in this report is a statement about this machine, not about GPUs in general.

```
(paste the make info panel here)
device      {...}          api {...}
subgroup    {N} lanes
timestamps  {...} ns per tick
statistics  pipeline statistics {supported / unsupported}
student.cpp  3 vertices, stride 20, 2 attributes
```

`{screenshot: info}`

Fixed for the whole lab: the render target is an off-screen **800 × 600** image, so **480,000 pixels** is the denominator behind every coverage figure below.

1. What I wrote

`student.cpp` is the only file I edited. Five TASKs, and the contract for all of them is `src/Utils/Student.h`.

TASK	What it is	Value I chose
1a	TRIANGLE	V0 (0, -0.5) red, V1 (-0.5, 0.5) green, V2 (0.5, 0.5) blue
1b	stride	20 bytes — <code>vec2 + vec3</code> with no padding, because vertex fetch reads exactly what the format and offset say
1c	attributes	<code>loc 0</code> → <code>R32G32_SFLOAT</code> at offset 0; <code>loc 1</code> → <code>R32G32B32_SFLOAT</code> at offset 8
2	<code>uniformBlock()</code>	<code>struct Params { mat4 mvp; uint costLoops; uint stripeWidth; uint _pad[2]; }</code> , 80 bytes
3	<code>pipelineState()</code>	CCW front face, back-face cull, <code>VK_COMPARE_OP_GREATER</code> , depth write on, shader and draw order picked from <code>Variant</code>
4	QUAD, QUAD_INDICES	four corners at ± 0.5 , indices <code>0 1 2 / 0 2 3</code>
5	<code>instanceBuffer()</code> , <code>recordDraw()</code>	100 × 50 grid of <code>translate · scale(0.01)</code> , one <code>vkCmdDrawIndexed(..., INSTANCES, ...)</code>

Two points worth stating because they are where the bytes actually are:

Winding. Facing is decided in framebuffer coordinates, where y points down. For `V0 (0, -0.5)`, `V1 (-0.5, 0.5)`, `V2 (0.5, 0.5)` the signed area of the Vulkan spec's rule comes out negative, which is the counter-clockwise case, and Part I culls back faces with front-facing set to `COUNTER_CLOCKWISE`. So this order is the front-facing one even though it looks counter-clockwise on paper only because paper has y up. Swapping `V1` and `V2` culls the triangle: I tried it and `rasterized` went to 0.0%. The same winding is reused for both triangles of `QUAD_INDICES`, which is why the square comes out solid rather than half.

Two layout rules for one type. `glm::vec3 color` occupies 12 bytes in the vertex buffer, and the same `vec3` would occupy 16 in a uniform block. TASK 1 lives on the vertex-fetch side of that line, TASK 2 on the std140 side: `mat4` at offset 0 size 64, `uint` at 64, `uint` at 68, block rounded up to a multiple of 16 → **80 bytes**, which is what `static_assert(sizeof(Params) == 80)` pins down and what the panel echoes.

`pipelineState()` and `Variant`. `Variant` is a four-valued `enum class`, not a struct, so both axes of the 2×2 experiment have to be recovered from the single value:

```
const bool earlyZ      = v == Variant::EarlyZFrontToBack ||
                        v == Variant::EarlyZBackToFront;
const bool frontToBack = v == Variant::EarlyZFrontToBack ||
                        v == Variant::WriteDepthFrontToBack;

s.fragShader = earlyZ ? "earlyz_a.frag" : "earlyz_b.frag";
s.drawOrder  = frontToBack ? DrawOrder::FrontToBack : DrawOrder::BackToFront;
```

`VK_COMPARE_OP_GREATER` is forced by the render pass, which clears depth to `0.0`: under Lab 05's reversed-Z convention the near plane is depth 1 and the far plane depth 0, so "keep the larger depth" is the test that lets anything pass at all against a 0.0 clear. `earlyz.vert` puts layer k at depth $(k+1)/(LAYERS+1)$, so layer 7 at 0.889 is the nearest and `DrawOrder::FrontToBack` correctly starts there.

2. Write-up 1 — Part I

{screenshot: part I – red apex top, green bottom left, blue bottom right}

```

vertex buffer  3 vertices, stride 20, offsets [0 8]
uniform block  80 bytes
rasterized    (...) % of the target          (must be 12.5%)
centroid      pixel (400, 350) reads ((r), (g), (b))
image         (...)
gpu time      (...) ms median, (...) ms p95

```

The 12.5% needs no tool: corners at ± 0.5 make a triangle of clip-space area 0.5 against a clip square of area 4.

1.1 Neither shader blends, yet the triangle has a gradient. Which stage?

Rasterization — the fixed-function stage sitting between the vertex shader and the fragment shader, box three of the five in Figure 1. It is the only stage in the pipeline that sees more than one vertex at a time, so it is the only one that *could* have produced the gradient.

For every covered sample it solves for the barycentric weights of that sample inside the triangle and evaluates Equation 1 on each `out` variable the vertex shader wrote:

$$f(P) = \lambda_0 \cdot f(V_0) + \lambda_1 \cdot f(V_1) + \lambda_2 \cdot f(V_2), \quad \lambda_0 + \lambda_1 + \lambda_2 = 1$$

on `fragColor` — three times, once per channel. The interpolation is perspective-correct, i.e. the weights are divided through by w per sample; here all three vertices leave the vertex shader with $w = 1$, so the correction is the identity and the result is plain affine interpolation across the triangle.

The fragment shader cannot be the source: by the time `main()` runs, `layout(location = 0) in vec3 fragColor` is already a single interpolated `vec3`. The shader never sees red, green and blue at once; it sees one colour per invocation and writes it straight out. Nothing blends because nothing needs to — the gradient was manufactured upstream, by fixed-function hardware, before any fragment shader existed.

In Lab 03 I did this by hand, in `{name the Lab 03 function – the barycentric weight computation inside your triangle filler}`: same equation, same three weights, computed on one CPU core per pixel instead of by the rasteriser.

1.2 Lab 05's `viewportTransform()` flipped y. Where did the flip go?

Into the convention, and therefore into fixed-function hardware.

Vulkan's clip space already points **y down**: after the perspective divide, $y = -1$ is the *top* row of the framebuffer and $y = +1$ the bottom. The viewport transform that the rasteriser applies is

$$y_f = (h / 2) \cdot y_d + (y_{\text{offset}} + h / 2)$$

with a **positive** h , so there is no negation anywhere in it. In Lab 05 I was working in the textbook/OpenGL convention where NDC y points up, so my `viewportTransform()` had to supply the negation itself to turn an upward y into a row index that grows downward.

Here that negation has been absorbed by the definition of clip space, so writing it a second time would flip the image, not fix it. That is exactly the trap in the check values: writing the apex at $y = +0.5$ out of OpenGL habit gives a triangle that points *down* — arithmetically correct, wrong convention. The apex having a **negative** y is the whole of the difference.

1.3 Centre-pixel value, measured and predicted

`triangle.cpp` samples the pixel at the centroid of `TRIANGLE`. The centroid is $((0 - 0.5 + 0.5)/3, (-0.5 + 0.5 + 0.5)/3) = (0, 1/6)$, which maps to pixel (400, 350) at 800×600 — and that is the pixel the panel names.

	R	G	B
Equation 1 predicts, $\lambda_0 = \lambda_1 = \lambda_2 = 1/3$	85	85	85
Measured	$\langle r \rangle$	$\langle g \rangle$	$\langle b \rangle$

$1/3 \cdot 255 = 85.0$ exactly in each channel: red contributes $1/3$ of 255 to R only, green $1/3$ of 255 to G only, blue $1/3$ of 255 to B only. The target is a UNORM format, so no sRGB curve sits between the shader's float and the value a colour picker reads — 85 in, 85 out.

A ± 1 discrepancy on one channel is expected rather than an error. The pixel *centre* at (400.5, 350.5) is not the exact centroid — the program truncates the centroid to integer pixel coordinates — so the sample's true λ are near $1/3$ but not equal to it, and the UNORM round of $255 \cdot \lambda$ can land one LSB away.

3. Write-up 2 — Part II, what did it cost?

Predictions were written into `predictions.md` before running anything. The instrument is the TASK 2 uniform buffer: `costLoops` is the only thing that changes between cells.

```
loops          0, 500, 2000 (ms median, in the vertex stage then the fragment stage)
1 triangle, small      (...) (...) (...) | (...) (...) (...)
1 triangle, fullscreen (...) (...) (...) | (...) (...) (...)
100k instances, small  (...) (...) (...) | (...) (...) (...)
rasterized           (...) % of the target
```

(screenshot: part II)

The invocation counts, and how I got the fragment ones

The handout lists vertex invocations and withholds fragment invocations. The placement in `cost.vert` is `pos = inPosition * scale + cell * (2/columns) - 1`, so each scene is pinned down by `scale`, `instances`, `columns` from `SCENES` in `cost.cpp`:

Scene	scale	inst	cols	coverage	fragment inv.	vertex inv.
S1 1 triangle, small	0.04	1	1	0.007%	36	3
S2 1 triangle, fullscreen	4.0	1	1	75.0%	360,000	3
S3 100k instances, small	0.04	100,000	317	100%	≈ 9,551,000	300,000

- **S2** is the one case the printed coverage gives directly. Its clipped vertices are $(-1, -3)$, $(-3, 1)$, $(1, 1)$; only the edge $y = 2x - 1$ cuts the square, and the wedge it removes has area $\int_0^1 2x \, dx = 1$ out of 4 $\rightarrow 75\%$, i.e. $0.75 \times 480,000 = \mathbf{360,000}$.
- **S1** is parked on the corner $(-1, -1)$, so only the quadrant inside the square survives: $\int_0^1 (y+1)/2 \, dy \div 2 = 0.375$ of a triangle whose full area is $0.5 \cdot 0.04^2 = 8.0 \times 10^{-4} \rightarrow 0.375 \cdot 8.0e-4 / 4 \cdot 480,000 \approx \mathbf{36 \, px}$.

- **S3 cannot be read off the coverage figure at all**, and this is the part of the exercise that has a trap in it. Each triangle is 16×12 px, but the grid pitch is $2/317$ clip units = 2.52×1.89 px, so the triangles overlap about twentyfold and coverage *saturates* at 100%. The fragment count has to come from summed area instead: $100,000 \times 8.0e-4 = 80$ clip units = $20 \times$ the clip square = 9,600,000, less the $\approx 0.5\%$ that falls off the right and top edges of the grid $\rightarrow \approx \mathbf{9,551,000}$.

I checked all three by rasterising the same geometry at 800×600 in a separate script: 36 / 360,000 / 9,551,435 fragments, coverage 0.007% / 75.000% / 100.000%, which is where the figures above come from.

2.1 Which scene is expensive in which placement?

	vertex-stage loop	fragment-stage loop
S1 small	3 × loops — floor	36 × loops — floor
S2 fullscreen	3 × loops — flat	360,000 × loops — steep
S3 100k instances	300,000 × loops — steep	9.55e6 × loops — steepest

Answer from the panel: {...}

The shape to look for: S1 is noise in both columns and should not move with `loops` at all — 3 and 36 invocations cannot be made expensive. S2 separates the stages cleanly: three vertices against 360,000 fragments, a ratio of 120,000, so its vertex row should be flat and its fragment row should rise roughly linearly (4× from loops 500 to 2000).

S3 is where I expect to contradict the handout's table, which calls it the scene that isolates the vertex stage. It does have $100,000\times$ more vertex work than S2 — but the $20\times$ overdraw also gives it $26.5\times$ more *fragment* work than S2. S3 isolates the vertex stage only in the sense of being the one scene with non-trivial vertex load; it is not fragment-cheap, and its fragment cell should be the largest number on the whole panel.

2.2 Same loop body in both files — explain the ratio between the two largest cells

The two shaders contain the identical statement `for (uint i = 0; i < u.costLoops; i++) { x = x * 1.000001 + 1.0; }`, so per iteration they cost the same c , and a cell's time is

$$t \approx t_{\text{baseline}} + (\text{invocations}) \times \text{loops} \times c$$

Everything except the invocation count cancels in a ratio, provided the loops = 0 baseline is subtracted first.

Predicted, largest against second largest:

$$t_{\text{S3,frag}} / t_{\text{S2,frag}} = 9.551 \times 10^6 / 3.60 \times 10^5 = \mathbf{26.5}$$

and, if the ordering comes out differently, the other candidate pairing is

$$t_{\text{S3,frag}} / t_{\text{S3,vert}} = 9.551 \times 10^6 / 3.0 \times 10^5 = \mathbf{31.8}$$

Measured: (largest) ms and (second) ms, both at loops = 2000, baselines {...} and {...} subtracted $\rightarrow$ ratio {...} against a predicted (26.5 / 31.8).

If the measured ratio comes in *below* the prediction, the honest reading is that the assumption "every rasterised fragment is shaded" failed, and on a tile-based GPU that is exactly what one expects: hidden-surface removal in S3 discards overlapped fragments before shading, so the effective fragment count is lower than the 9.55×10^6 that geometry alone predicts. That would make the *discrepancy itself* the measurement of how much of the 20× overdraw the hardware removed for free.

2.3 The smallest cell is near zero — what is the smallest difference I can resolve?

Two separate floors, and the larger of the two wins.

1. **Quantisation.** `./info` reports `<...>` ns per timestamp tick, and `Timer.cpp` brackets the pass with two `vkCmdWriteTimestamp` queries, so one pass cannot be resolved finer than one tick — `<...>` ms.
2. **Dispersion**, which dominates. `timePass` runs `WARMUP_FRAMES = 5` then `MEASURED_FRAMES = 21` and reports median and p95. The gap between them on the *cheapest* cell is the run-to-run noise of an empty pass, and no difference smaller than that is a measurement: median `<...>` ms, p95 `<...>` ms → floor $\approx$ `<...>` ms.

How I know: the S1 cells at loops = 0, 500 and 2000 differ by exactly 6.0×10^3 and 7.2×10^4 loop iterations, which is real work — so any spread I see across those three cells is noise rather than signal, and it puts a number on the floor directly.

4. Write-up 3 — Part III, how wide is a warp in pixels?

`stripes.vert` draws `TRIANGLE × 4.0` centred (no corner offset), so the triangle's clipped area is $4 - 0.5 = 3.5$ of 4 → **87.5% coverage = 420,000 fragments**, with `COST` fixed at 2000. The only thing the sweep changes is *which pixels take which branch*, and both branches compile to the same instruction sequence (`× 1.000001` against `× 1.000002`).

```
stripe    1 px  <...> ms median
stripe    2 px  <...>
stripe    4 px  <...>
stripe    8 px  <...>
stripe   16 px  <...>
stripe   32 px  <...>
stripe   64 px  <...>
stripe  128 px  <...>
no branch      <...>
rasterized    87.5% of the target
```

{screenshot: part III}

3.1 Plot, and the width at which the penalty disappears

{plot: stripe width on a $\log_2$ x-axis against ms median, with the no-branch run as a horizontal reference line}

The penalty disappears at **(w) px**, where "disappears" means the curve has come down to within the noise floor of the no-branch run.

3.2 Does w equal N , $\sqrt{N}$, or neither?

`./info` reports a subgroup of **(N) lanes**. With $\langle N \rangle = 32$ the three candidates are 32, $\sqrt{32} = 5.66$, and the measured $\langle w \rangle$.

Expected answer: **neither**. A subgroup of N fragment lanes does not cover a $1 \times N$ row of pixels, and it is not square either — fragment shading is dispatched in 2×2 quads (that is what makes screen-space derivatives possible), and N of them are packed into a rectangle $w \times h$ with $w \cdot h = N$. Divergence along x therefore disappears once one stripe is at least as wide as the **x extent** of that rectangle, not as wide as the subgroup:

Layout of 32 lanes	w	penalty gone at
8×4 (Apple, NVIDIA)	8	8 px
16×2	16	16 px
1×32 (does not occur)	32	32 px

So a measured $w = 8$ with $N = 32$ says the pixel group my GPU shades together is **8 pixels wide and 4 tall** — a compact tile chosen to keep derivatives and texture locality cheap, not a scanline. Reading $w = \sqrt{N}$ would require a 5.66×5.66 group, which cannot be built from 2×2 quads at all.

One caveat the measurement must respect: stripes are cut by `uint(gl_FragCoord.x) / stripeWidth`, so the boundaries are aligned to multiples of the width. A group whose x origin is also a multiple of w sees the transition exactly at `stripeWidth = w`; a misaligned one can still straddle a boundary at w and only converge at $2w$, which shows up as a knee spread over two points rather than a single step.

3.3 What Equation 2 predicts for the 1-pixel case

$$\text{effective throughput} = (\text{active lanes} / \text{subgroup size}) \times \text{peak throughput}$$

At `stripeWidth = 1`, alternate pixel columns take opposite branches. Any group at least 2 px wide therefore contains both, so the hardware must execute **both** branches, masking off the lanes not taking each one. With 32 lanes split evenly, 16 are active in each pass:

$$16 / 32 = 0.5 \implies \text{effective} = 0.5 \times \text{peak} \implies t_{1\text{px}} = 2.0 \times t_{\text{no branch}}$$

	value
Equation 2 predicts	$2.00 \times$ the no-branch run
Measured	$\langle t_1 \rangle / \langle t_{\text{flat}} \rangle = \langle \dots \rangle \times$

If they disagree, the plausible reasons, in the order I would check them:

- **Ratio well below 2×**. The two branches were *if-converted*: both are three arithmetic ops, so the compiler can evaluate each and select, which removes the divergence and the measurement along with it. `costLoops` is the knob that prevents this — see 3.4.
- **Ratio below 2× but the curve is still clearly shaped**. The fixed cost of the pass (rasterising 420,000 fragments, the `clamp` tail, the render-pass load/store) is in both numbers and dilutes the ratio. Equation 2 governs the *loop* portion only, so the baseline has to come out before the ratio goes in.

- **Ratio above 2x.** Occupancy fell for a reason unrelated to masking — the two-branch version needs more live registers, so fewer warps fit per SM and latency hiding gets worse on top of the lane masking.

3.4 A flat curve everywhere means costLoops is too small

`stripes.cpp` hard-codes `COST = 2000`, so raising it means editing that constant (it is not exposed through `ARGS`). The value at which the curve appeared: `{...}`. Below that value the loop is short enough that the compiler turns both branches into straight-line arithmetic with no branch left to diverge on, and every stripe width measures the same thing.

5. Write-up 4 — Part IV, early-Z and draw order

Eight copies of the triangle at `COVERAGE = 4.0`, perfectly overlapping, with Part II's expensive fragment shader at `COST = 2000`. `earlyz.vert` puts layer k at depth $(k+1)/9$, depth is cleared to 0.0 and compared with `GREATER`, so larger depth = nearer.

The two shader variants differ in one line. `earlyz_a.frag` declares `layout(early_fragment_tests) in;` — the depth test may run before the shader. `earlyz_b.frag` writes `gl_FragDepth = gl_FragCoord.z`, which is numerically the same depth, but the moment a shader *can* write depth the test cannot be hoisted in front of it. Same image, different amount of work.

```
layers          8 copies, 2000 loops each
early-Z         front-to-back  {...} ms median, {...} ms p95
early-Z         back-to-front  {...} ms median, {...} ms p95
writes depth    front-to-back  {...} ms median, {...} ms p95
writes depth    back-to-front  {...} ms median, {...} ms p95
rasterized      87.5% of the target
```

(screenshot: part IV)

4.1 The 2 × 2 table

ms median	front-to-back	back-to-front
early-Z (<code>earlyz_a.frag</code>)	<code>{...}</code>	<code>{...}</code>
writes depth (<code>earlyz_b.frag</code>)	<code>{...}</code>	<code>{...}</code>

4.2 Fragment shader invocations, slowest and fastest cell

Coverage is 87.5% of 480,000 = **420,000 pixels per layer**, and all eight layers cover exactly the same pixels because they share one scale.

Cell	early-Z possible?	rejected before shading?	invocations
early-Z, front-to-back	yes	yes — layers 2...8 fail GREATER	420,000
early-Z, back-to-front	yes	no — every layer is nearer than the last, all pass	3,360,000
writes depth, front-to-back	no	no — late test discards <i>after</i> shading	3,360,000
writes depth, back-to-front	no	no	3,360,000

- **Slowest cell: $8 \times 420,000 = 3,360,000$ invocations.** Three of the four cells land here, for two different reasons: `earlyz_b` loses the early test outright, and `earlyz_a` back-to-front keeps the test but never fails it, because drawing far-to-near means every new layer is in front of what is already there.
- **Fastest cell: 420,000 invocations**, the nearest layer only — it avoids $7 \times 420,000 = 2,940,000$ invocations, 87.5% of the work.

The asymmetry is the whole point: early-Z is worth nothing without the draw order, and the draw order is worth nothing without early-Z. Only the cell that has both is cheap.

4.3 A row that barely changes with draw order

Device: **{device name from .info}**, which is a {tile-based / immediate-mode} GPU.

If the two early-Z columns come out close, the reason is architectural rather than a bad measurement. A tile-based GPU — Apple silicon among them — bins primitives per tile and resolves visibility for the whole tile on chip *before* dispatching any fragment shading, so it removes hidden surfaces whatever order I submitted them in. On such a GPU, `earlyz_a` back-to-front gets most of the benefit that the front-to-back submission was supposed to earn, and my prediction of 3,360,000 invocations for that cell is wrong in the hardware's favour.

What it did to my table: {...}. The `earlyz_b` row is the control — writing `gl_FragDepth` defeats the on-chip resolve as well, so if the `earlyz_b` row is the slow one and is flat across draw order, that is the signature of tile-based hidden-surface removal rather than of a broken experiment.

6. Write-up 5 — Part V, the index buffer

Computed before measuring, as the handout asks.

```

unindexed    6 vertices x 20 B = 120 bytes
indexed      4 x 20 B + 6 x 2 B = 92 bytes
saving       23.3%
index type    VK_INDEX_TYPE_UINT16
rasterized    {...}% of the target          (must be 25.0%)
image         {...}
gpu time      {...} ms median, {...} ms p95

```

{screenshot: part V}

6.1 The two byte totals and the saving

	arithmetic	bytes
Six vertices, no index buffer	6×20	120
Four vertices plus six <code>uint16_t</code>	$4 \times 20 + 6 \times 2$	92
Saving	$1 - 92/120$	23.33%

The square covers 25.0% of the target: ± 0.5 in both axes is one unit square out of the 2×2 clip square.

6.2 Extrapolating to the cube and sphere of 6.2.2

The general form falls out immediately. With I indices, V distinct vertices, a vertex of s bytes and an index of b bytes:

$$\text{saving} = 1 - (V \cdot s + I \cdot b) / (I \cdot s) = 1 - 1/r - b/s, \quad r = I / V$$

Two terms, and they say everything. $1/r$ is the share of the bytes that the index buffer can never remove, set purely by how often a vertex is reused; b/s is what the index buffer costs to have. The ceiling, as $\text{reuse} \rightarrow \infty$, is $1 - b/s = \mathbf{90\%}$ for a 20-byte vertex and a 16-bit index.

Mesh	I	V	r	saving, s = 20	saving, s = 32
My square	6	4	1.5	23.3%	27.1%
Cube	36	8	4.5	67.8%	71.5%
Sphere, 32×16 grid	3,072	561	5.48	71.7%	75.5%

Do they agree with the chapter's 73–83% claim? Not from a 20-byte vertex — 67.8% and 71.7% both fall short. They agree once the vertex is fattened: a 32-byte vertex (position + normal + uv) gives 71.5% and 75.5%, and pushing to a 48-byte vertex with tangents puts the sphere at 78%. So the chapter's range is a statement about *its* vertex format as much as about its meshes, and the missing variable in the claim is s .

What is different about my square is r , not s : 6 indices over 4 vertices is a reuse ratio of 1.5, against 4.5 for a cube and 5.5 for that sphere. Two triangles sharing one diagonal is nearly the worst case an index buffer can be handed — $1/r = 0.667$ of the bytes are irreducible before the index buffer even costs me its 0.1. Indexing pays in proportion to how many triangles meet at the average vertex, and a square has almost none.

6.3 Was the indexed square any faster?

	gpu time (ms median)
Unindexed reference	{...}
Indexed	{...}

Expected: **no measurable difference**, and the reason is scale. This frame shades 25% of $480,000 = \mathbf{120,000}$ **fragments** against a saving of 28 bytes and at most two vertex-shader invocations. 28 bytes is not a memory system's rounding error, it is far below it — a single cache line is 64 or 128 bytes, so both versions are one fetch either way.

What the 23.3% actually measures is **footprint**, not time: bytes resident in device memory, bytes crossing the PCIe bus or the unified-memory fabric at upload, and bytes occupied in the vertex cache. Those matter when the mesh is the sphere rather than the square and the scene holds thousands of them. The second thing indexing buys — and the reason it is universal in practice — is that a shared vertex becomes eligible for the post-transform cache, so vertex *shading* is done once per distinct vertex rather than once per corner. At three of my six corners that is unmeasurable; at 5.48 reuse it is a real 5× reduction in vertex work.

7. Write-up 6 — Part VI, name the bottleneck, prove the fix

Three scenes, none labelled, each slow for a different reason. `probe.cpp` sweeps each one three ways — halve the pixels (`scale × 0.7071`, so area × 0.5), halve the instances, halve the bytes fetched per fragment — and prints the CPU frame time beside the GPU pass time.

```
storage      320000 bytes, stride 64
              baseline half px half inst half bytes  cpu
scene 1      (...)      (...)      (...)      (...)      (...)
scene 2      (...)      (...)      (...)      (...)      (...)
scene 3      (...)      (...)      (...)      (...)      (...)
5000 draws   gpu (...) ms, cpu (...) ms
1 draw       gpu (...) ms, cpu (...) ms
rasterized   (...) % of the target
```

(screenshot: part VI)

320,000 bytes is the check value for TASK 5: in `std430` an array of `mat4` has a stride of 64, so $5,000 \times 64 = 320,000$ exactly.

7.1 Classifying the three scenes

From `SCENES` in `probe.cpp` and the geometry of `QUAD` through `instanceBuffer()`:

Scene	scale	inst	fetches	loops	fragments	deciding sweep	verdict
1	2.0	1	1	3000	480,000 (100%)	halves with pixels, identical in bytes	fragment ALU
2	2.0	1	64	0	480,000 (100%)	halves with pixels and with bytes	bandwidth
3	0.6	5,000	1	0	20,000 (4.2%)	flat in pixels, halves with instances, CPU ≫ GPU	submission

Measured, with the numbers that decided each one:

- **Scene 1 → (verdict).** `(baseline)` → `(half px)` on the pixel sweep, a factor of `(...)` against the 2.0 that halving 480,000 fragments predicts, while the byte sweep gives `(half bytes)`. Note that scene 1's byte sweep is a *literal re-run*: `fetches` is 1 and $\max(1/2, 1) = 1$, so an unchanged number there is the control, not a finding.
- **Scene 2 → (verdict).** The distinguishing evidence is that **both** sweeps move: `(baseline)` → `(half px)` and `(baseline)` → `(half bytes)` ($64 \rightarrow 32$ `vec4` fetches per fragment). Scene 1 moves with pixels only. Halving pixels halves the *number* of fetches and halving `fetches` halves the fetches *per fragment* — a scene limited by arithmetic would ignore the second.

- **Scene 3 → (verdict).** 20,000 fragments is 1/24 of scene 1's, so the pixel sweep should be flat; the instance sweep halves it because it halves the number of `vkCmdDrawIndexed` calls, and the `cpu` column exceeding the GPU median is the direct statement that the GPU is waiting for the CPU.

7.2 TASK 5 before and after

	gpu (ms median)	cpu (ms)
5,000 separate <code>vkCmdDrawIndexed</code>	{...}	{...}
1 instanced <code>vkCmdDrawIndexed(..., 5000, ...)</code>	{...}	{...}

The CPU was the bottleneck before the fix, and the `cpu` column says so — it sits above the GPU median in the "5000 draws" row, which means the queue drained faster than the CPU could refill it.

The GPU numbers should be close to unchanged, and that is the point rather than a disappointment: the same 5,000 quads, the same 30,000 vertex invocations, the same 20,000 fragments reach the hardware either way. What changed is 5,000 command-buffer records and 5,000 driver-side validation and state-setup paths collapsing into one, with `gl_InstanceIndex` doing in the vertex shader what `firstInstance` was doing in 5,000 separate calls. A fix that moves only the CPU number is the correct outcome for a submission-bound scene, and it is also the proof that the classification was right.

7.3 One captured frame

{screenshot: Xcode GPU trace of one frame}

Three most expensive events: {1}, {2}, {3}.

One note on the mechanics: `probe.cpp`'s `main()` takes no arguments, so the `--capture` that `make probe ARGS=--capture` forwards is read by nothing in this skeleton. On macOS the capture has to be armed from outside the program — the MoltenVK environment variables that open a Metal capture scope, then Xcode's GPU frame capture — since RenderDoc does not support macOS at all.

7.4 SM occupancy on an H100, 48 registers per thread

No hardware needed. An H100 SM has a register file of 65,536 32-bit registers and can hold at most 64 resident warps. Registers are allocated per warp, so a warp of 32 threads at R registers each locks up $32 R$ registers, and the register file caps the number of resident warps at $\lfloor 65,536 / 32R \rfloor$.

$$32 \times 48 = 1,536 \text{ registers per warp}$$

$$\lfloor 65,536 / 1,536 \rfloor = \lfloor 42.67 \rfloor = 42 \text{ resident warps}$$

$$\text{occupancy} = 42 / 64 = \mathbf{65.6 \%}$$

The floor matters: 42.67 warps cannot exist, and the 0.67 of a warp's worth of registers — 1,024 of them — simply sits idle.

The two anchors in the question confirm the method rather than being assumed by it:

R	regs/warp	resident warps	occupancy
32	1,024	$65,536 / 1,024 = 64$	$64 / 64 = 100\%$ ✓
48	1,536	$\lceil 42.67 \rceil = 42$	65.6%
64	2,048	$65,536 / 2,048 = 32$	$32 / 64 = 50\%$ ✓

Reading across the table: occupancy is a step function of register pressure, not a smooth one, and the steps are where $65,536/32R$ crosses an integer. Going from 32 to 48 registers costs 34 percentage points of latency hiding for 16 registers, which is why register pressure is a first-class optimisation target and why 32 registers per thread is such a common design point.

8. AI prompt log

One entry per prompt: the prompt, the model, the idea behind it — what I needed and what I already knew — and the verification.

Entry 1

Prompt	"แก้ student.cpp ที่ทำให้โค้ด run ไม่ได้หน่อย" (fix what makes student.cpp fail to build)
Model	Claude Opus 5 (Cowork)
Idea	I had already written all five TASKs and <code>make</code> failed inside <code>pipelineState()</code> . What I knew: the six field names live in <code>src/core/Pipeline.h</code> and <code>Variant</code> selects both axes of the Part IV experiment. What I did not know was which of my names were wrong, so I wanted the mismatch located against the header rather than the function rewritten for me.
Verification	Read <code>src/core/Pipeline.h</code> myself and confirmed all three renames (<code>depthCompareOp</code> → <code>depthCompare</code> , <code>depthWriteEnable</code> → <code>depthWrite</code> , <code>fragmentShader</code> → <code>fragShader</code>) and that <code>Variant</code> is a four-valued <code>enum class</code> , so my <code>v.variant / v.frontToBack / Variant::A</code> could never have compiled. Confirmed <code>earlyz_a.frag</code> is the <code>early_fragment_tests</code> variant and <code>earlyz_b.frag</code> the one writing <code>gl_FragDepth</code> , so the <code>earlyZ</code> flag maps to the right file. Rebuilt with the project's own flags (<code>-std=gnu++20 -Wall -Wextra</code>) — clean, no warnings. Then <code>make earlyz</code> and checked that <code>fragShader</code> is non-empty (the panel's TASK 3e check) and that all four variants produce different timings rather than four identical numbers.

Entry 2

Prompt	"do the report for me (using instruction file as guideline) and see prompts in Lab explanation"
Model	Claude Opus 5 (Cowork)
Idea	The six write-ups each mix a measured number with a derivation, and I wanted the derivations laid out from the actual source — <code>cost.vert</code> 's placement formula, <code>SCENES</code> , <code>COVERAGE</code> , <code>instanceBuffer()</code> — rather than from a general description of early-Z or of index buffers. What I already knew: every measured figure has to come from my own machine, so the report was always going to be a frame I fill in, not a finished artefact.
Verification	Re-derived each geometric figure by hand before accepting it, and then independently rasterised the same geometry at 800×600 in a separate script: 36 / 360,000 / 9,551,435 fragments for the three <code>cost</code> scenes, 87.5% for <code>stripes / earlyz</code> , 25.0% for the indexed square, 4.17% for probe scene 3. Cross-checked the byte totals against <code>index.cpp</code> 's own <code>reportBytes()</code> (120, 92, 23.3%) and against the handout's stated check values. Checked the occupancy method against the two anchors the question supplies — 32 registers → 100%, 64 → 50% — before trusting it at 48. (After the runs: confirmed each (...) against the panel and flagged where the measurement disagreed with the prediction.)

Entry 3 onwards

(one entry per further prompt – keep the idea and verification fields filled; an entry with those blank does not meet the requirement)

9. Submission checklist

- [] `student.cpp` — five TASKs, builds clean with `-Wall -Wextra`
- [] `predictions.md` — Part II's six cells, written before measuring
- [] `make info` screenshot, device name, subgroup size, timestamp period
- [] Part I triangle screenshot
- [] Panels for Parts II–VI
- [] Six write-ups, every (...) replaced
- [] One GPU frame capture with its three most expensive events
- [] AI prompt log, every entry with idea and verification filled in